

LAB-07 Detailed Notes

LAB-07: Integrating Multiple UVCs

1. Objective of Lab-07

The objective of this lab is to integrate multiple UVCs into a single reusable verification environment and verify communication between them.

Previously in Lab-06:

YAPP Packet Source -> Router DUT

Only the YAPP UVC was connected.

In Lab-07, we add:

Router DUT with YAPP UVC, CHANNEL UVC, HBUS UVC.

This allows:

- Packet generation from YAPP interface
- Packet observation on output channels
- Router register programming through HBUS

2. UVCs Used in Lab-07

2.1 YAPP UVC

Purpose: Generates router input traffic

Components:

yapp_tx_agent

- Driver
- Sequencer
- Monitor

Responsibilities:

- Generate packets
- Drive packets into DUT
- Monitor DUT input

2.2 Channel UVC

Purpose: Monitor packets leaving router outputs

Three instances used:

- Channel 0
- Channel 1
- Channel 2

Each instance contains:

channel_env

- rx_agent
- sequencer
- driver
- monitor

2.3 HBUS UVC

Purpose: Configure router registers

Used to:

- Program MAXPKTSIZE
- Program ROUTER_EN
- Read router registers

3. Router Register Map

MAXPKTSIZE Register

Address: 0

Width: 8 bits

Default Value: 63 (0x3F)

Purpose:

Maximum packet payload size accepted by router

Examples:

MAXPKTSIZE = 20

Allowed: length = 1..20

Rejected: length > 20

ROUTER_EN Register

Address: 1

Width: 1 bit

Default: 1

Purpose:

Enable/Disable Router

4. New Environment Architecture

Before Lab-07:

base_test

+-- yapp_env

After Lab-07:

base_test

+-- router_tb

+-- env (YAPP)

+-- env1 (Channel 0)

+-- env2 (Channel 1)

+-- env3 (Channel 2)

+-- henv (HBUS)

5. router_tb

Main integration component.

Purpose:

Instantiate all UVC environments

Components Created

YAPP Environment

yapp_env env;

Purpose: Packet source environment

Channel Environments

```
channel_env env1;
```

```
channel_env env2;
```

```
channel_env env3;
```

Purpose: Output packet collection

HBUS Environment

```
hbus_env henv;
```

Purpose: Router configuration interface

6. Channel Environment

Structure:

```
channel_env
```

- tx_agent

- rx_agent

- monitor

```
has_tx
```

Purpose: Enable transmitter agent

Configured:

```
set_config_int("env1","has_tx",0);
```

Meaning:

Do not create TX agent. Only RX side needed.

```
has_rx
```

Default: 1

Meaning: Create receiver side.

7. top_dut Modifications

Channel Interfaces Added

```
channel_if chn1(clock,reset);
```

```
channel_if chn2(clock,reset);
```

```
channel_if chn3(clock,reset);
```

Purpose:

Connect DUT output ports to Channel UVCs

HBUS Interface Added

```
hbus_if hif(clock,reset);
```

Purpose:

Connect DUT register interface

8. DUT Connections

Channel 0

```
.data_0(chn1.data)
```

```
.data_vld_0(chn1.data_vld)
```

```
.suspend_0(chn1.suspend)
```

Purpose: Router Output Channel 0

Channel 1

```
.data_1(chn2.data)
.data_vld_1(chn2.data_vld)
.suspend_1(chn2.suspend)
```

Channel 2

```
.data_2(chn3.data)
.data_vld_2(chn3.data_vld)
.suspend_2(chn3.suspend)
```

HBUS Connections

```
.haddr(hif.haddr)
.hdata(hif.hdata)
.hen(hif.hen)
.hwr_rd(hif.hwr_rd)
```

Purpose:

Register access interface

9. Virtual Interface Configuration

YAPP Interface

```
yapp_vif_config::set(null,"*", "vif",in0);
```

Purpose:

Provide DUT interface to YAPP UVC

HBUS Interface

```
hbus_vif_config::set(null,"*", "vif",hif);
```

Purpose:

Provide HBUS interface to HBUS UVC

Channel Interfaces

```
channel_vif_config::set(null,"*.env1*", "vif",chn1);
channel_vif_config::set(null,"*.env2*", "vif",chn2);
channel_vif_config::set(null,"*.env3*", "vif",chn3);
```

Purpose:

Associate each Channel UVC with a specific DUT output channel

10. simple_test

Purpose:

Run a complete multi-UVC test

Type Override

```
yapp_packet::type_id::set_type_override(short_yapp_packet::get_type());
```

Purpose:

Generate short packets only

YAPP Default Sequence

```
yapp_012_seq
```

Generates:

Packet Address 0

Packet Address 1
Packet Address 2

Channel Response Sequence
channel_rx_resp_seq

Purpose:
Continuously acknowledge packets

Without this sequence:
Router output FIFOs never drain

11. Packet Routing Verification

Address = 0
Expected: Channel 0
Observed: Channel_0 Packet Collected
PASS

Address = 1
Expected: Channel 1
Observed: Dropped because length=0
PASS

Address = 2
Expected: Channel 2
Observed: Channel_2 Packet Collected
PASS

12. Major Debugging Issues Faced

Issue 1
Virtual Interface Not Found

Error:
NOVIF

Cause:
Wrong wildcard path

Wrong:
.chan0

Correct:
.env1

Issue 2
Channel RX Sequence Not Starting

Wrong:
router.env1.agent.sequencer

Correct:
router.env1.rx_agent.sequencer

Reason:
channel_env contains rx_agent not agent

Issue 3
Channel Monitors Collected 0 Packets

Cause:
channel_rx_resp_seq not running

Result:
Router output FIFOs not drained

13. Final Verification Flow

yapp_012_seq
-> YAPP Driver
-> Router DUT
-> addr=0 -> Channel 0
-> addr=1 -> Channel 1
-> addr=2 -> Channel 2

HBUS Interface
-> Register Programming

Lab-07 Key Learning Outcomes

1. Multi-UVC integration.
2. UVM configuration database usage.
3. Virtual interface configuration.
4. Multi-agent environment construction.
5. Packet routing verification.
6. Channel response sequencing.
7. Hierarchical UVM path debugging.
8. Register configuration through HBUS.
9. Factory overrides in multi-UVC environments.
10. Preparing the environment for Virtual Sequencers and Virtual Sequences in Lab-08.

Master Revision Summary

Lab-07 Goal:
Integrate YAPP + Channel + HBUS UVCs into a single router verification environment.

Main Components:

router_tb
- yapp_env
- channel_env(0)
- channel_env(1)
- channel_env(2)
- hbus_env

Key Features:

- ✓ Short packet override
- ✓ Channel response sequences
- ✓ HBUS integration
- ✓ Virtual interface configuration
- ✓ Packet routing verification
- ✓ Multi-UVC coordination

Major Debug Lessons:

- ✓ NOVIF debugging
- ✓ UVM hierarchy path debugging
- ✓ Config DB wildcard matching
- ✓ Sequencer path corrections
- ✓ Channel FIFO draining via response sequences

Foundation for Lab-08:

Virtual Sequencer

Virtual Sequence

Cross-UVC coordination

HBUS + YAPP integrated test scenarios